DATA ENGINEERING

Data Partitioning Visual Guide

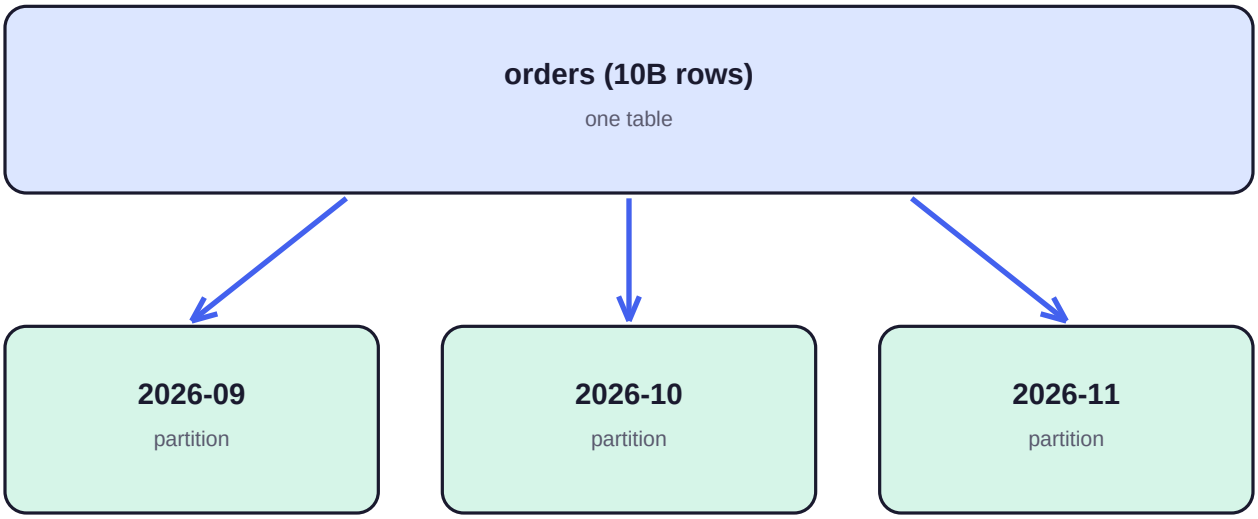
Divide data to conquer scale.
Strategies, pruning, and skew.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Why Partition?

One giant table is one giant problem. Partitioning divides it into manageable pieces.



Partitions are independent units: query one, drop one, compact one — without touching the rest. Operations scale with the partition, not the table.

STEP 2/6

Partitioning Strategies

Range, hash, and list — pick by query pattern.

Range

By value intervals: date, id ranges. Great for time queries.

Hash

hash(key) mod N: even spread, no ordering. Great for point lookups.

List

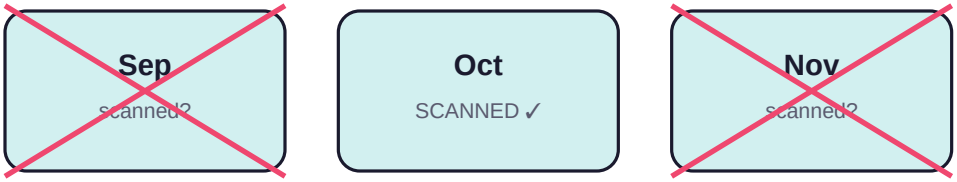
Explicit value sets: region = 'EU'. Great for known categories.

Composite: range by date, then hash by id within each — time pruning plus even distribution. Most large tables end up composite.

STEP 3/6

Partition Pruning

The optimizer skips partitions that can't match — the whole point.




```
WHERE order_date BETWEEN '2026-10-01' AND '2026-10-31'
```

Pruning only works when the filter is on the partition key with a sargable predicate. Filter on a non-partition column and every partition gets scanned.

- Partition key should be your most common filter
- Avoid functions on the partition column: WHERE year(d) = 2026 kills pruning
- Check EXPLAIN: partitions scanned should be << total partitions

STEP 4/6

Skew: The Partition Killer

One giant partition ruins parallelism — every worker waits for the slowest.

Hash partitioning on a low-cardinality or Zipf-distributed key creates hot partitions. Fixes:

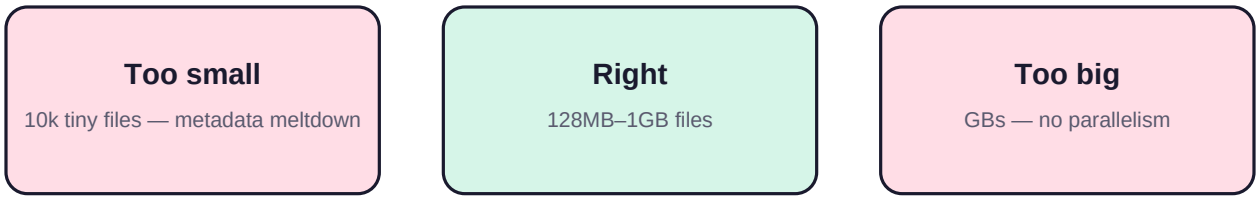
- Salting: append random suffix to hot keys, split across sub-partitions
- Composite keys: (region, user_id) instead of region alone
- Isolate the hot key: its own partition + special handling

Measure skew first: rows per partition. If the max is 10x the median, you have a skew problem regardless of strategy.

STEP 5/6

Partition & File Sizing

Right-sized partitions keep the engine happy.



Too many partitions: the Hive metastore / file listing becomes the bottleneck. Too few: no pruning benefit. Aim for partitions ≥ 1GB with a few files each.

- Don't partition by high-cardinality columns (user_id with 100M values)
- Date + one low-cardinality key is the sweet spot for most tables
- Compaction (OPTIMIZE) merges small files within partitions

STEP 6/6

Hidden Partitioning (Iceberg-style)

Partition by transform, query by column — no more partition-column gymnastics.

```
PARTITIONED BY (days(order_ts))   -- you write:

WHERE order_ts >= '2026-10-01'     -- no partition column!
```

Iceberg/Delta map your filter to partitions automatically. Evolve partitioning later (days → hours) without rewriting data or queries.

- No more year=/month=/day= path rituals in queries
- Partition evolution: change layout without breaking readers
- The table format handles the mapping — you think in columns

STEP ✓I✓

Key Takeaways

Data Partitioning — Remember

- Partitioning splits tables into independent units — operations scale with the partition, not the table.
- Range for time/ordered queries, hash for even spread, list for known categories; composite for both.
- Partition pruning is the payoff: filter on the partition key or every partition gets scanned.
- Skew kills parallelism — salt hot keys, use composite keys, measure rows per partition.
- Size partitions ≥ 1GB with few files each; hidden partitioning (Iceberg) removes the path rituals.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev